PROBLEM STATEMENT 10: Fleet Maintenance & Fuel Consumption Monitoring System

---

1. Hackathon Objective

This hackathon evaluates students on:

- *Java OOP design*
- *Spring Boot REST API development*
- *SQL database modeling*
- *Workflow state management*
- *Preventive maintenance scheduling*
- *Business rule enforcement*
- *Role-based responsibilities*
- *Fuel consumption analytics*
- *Cost tracking and budget management*

---

2. Problem Statement

Design and develop a Backend Fleet Maintenance & Fuel Consumption Monitoring System for a transportation and logistics company.

The system should allow:

- *Fleet Managers to register vehicles and schedule maintenance*
- *Drivers to log fuel refills and report issues*
- *Mechanics to perform maintenance and record repairs*
- *System to auto-schedule preventive maintenance based on mileage*

- *System to calculate fuel efficiency (km/l)*
- *Admin to monitor fleet health and generate cost reports*
- *Track complete vehicle lifecycle from acquisition to decommissioning*

The system must enforce preventive maintenance schedules and track all costs.

---

3. Core Domain Entities

3.1 User

- *id*
- *name*
- *email*
- *role*
- *createdAt*

Roles:

- *ADMIN*
- *FLEET_MANAGER*
- *DRIVER*
- *MECHANIC*

3.2 Vehicle

- *id*
- *registrationNumber*
- *vehicleType (TRUCK, BUS, VAN, CAR)*
- *make (TATA, ASHOK_LEYLAND, VOLVO, MARUTI)*

- *model*
- *yearOfManufacture*
- *fuelType (DIESEL, PETROL, CNG, EV)*
- *odometerReading*
- *status*

Vehicle Status Lifecycle:
ACTIVE → UNDER_MAINTENANCE → RETIRED

3.3 MaintenanceSchedule

- *id*
- *vehicle*
- *scheduleType (PREVENTIVE, BREAKDOWN, INSPECTION)*
- *serviceIntervalKM (default: 5000)*
- *lastServiceKM*
- *nextServiceDueKM*
- *status (SCHEDULED, IN_PROGRESS, COMPLETED, OVERDUE)*

3.4 MaintenanceRecord

- *id*
- *vehicle*
- *performedBy (User - MECHANIC)*
- *maintenanceDate*
- *serviceKM*
- *serviceType (OIL_CHANGE, TIRE_ROTATION, ENGINE_REPAIR, BRAKE_SERVICE, GENERAL)*
- *cost*

- *partsReplaced*
- *remarks*

3.5 FuelLog

- *id*
- *vehicle*
- *driver (User - DRIVER)*
- *fuelAmountLiters*
- *fuelCost*
- *odometerAtRefill*
- *fuelType (DIESEL, PETROL)*
- *refillDate*
- *receiptUrl*

3.6 IssueReport

- *id*
- *vehicle*
- *reportedBy (User - DRIVER)*
- *issueType (ENGINE, BRAKE, ELECTRICAL, TIRE, TRANSMISSION, OTHER)*
- *severity (LOW, MEDIUM, HIGH, CRITICAL)*
- *description*
- *reportedAt*
- *resolvedAt*
- *resolvedBy*

3.7 VehicleCost

- *id*
- *vehicle*
- *costType (MAINTENANCE, FUEL, INSURANCE, TAX, TIRE, ACCIDENT)*
- *amount*
- *recordedAt*
- *remarks*

3.8 VehicleActivityLog

- *id*
- *vehicle*
- *activity*
- *performedBy*
- *oldStatus*
- *newStatus*
- *timestamp*

Vehicle Status Lifecycle

ACTIVE → UNDER_MAINTENANCE → RETIRED

Workflow Rules

- *Vehicle must be ACTIVE to assign to driver*
- *Preventive maintenance every 5,000 km*

- *Maintenance OVERDUE if 500 km past due*
- *CRITICAL issues require immediate maintenance*
- *Vehicle cannot be ACTIVE if UNDER_MAINTENANCE*
- *Fuel log must show increasing odometer reading*
- *Each maintenance record must have cost*
- *RETIRED vehicles cannot be assigned*

---

4. MUST HAVE REQUIREMENTS

4.1 User Management

- *Create user with role*
- *Fetch users by role*
- *Fetch all users (ADMIN only)*

4.2 Vehicle Management

FLEET_MANAGER can:

- *Register vehicle (ACTIVE)*
- *Update odometer reading*
- *Retire vehicle (RETIRED)*
- *View all vehicles*

ADMIN can:

- *View fleet summary*

4.3 Maintenance Management

FLEET_MANAGER can:

- *Create maintenance schedule*
- *Assign vehicle for maintenance (UNDER_MAINTENANCE)*
- *Approve maintenance completion*

SYSTEM can:

- *Auto-schedule maintenance based on mileage*
- *Calculate next service due date*
- *Flag overdue maintenance*

MECHANIC can:

- *Record maintenance performed*
- *Add parts replaced*
- *Update vehicle status to ACTIVE after completion*

4.4 Fuel Management

DRIVER can:

- *Log fuel refill (with odometer)*
- *View fuel history of assigned vehicle*

SYSTEM can:

- *Calculate fuel efficiency (km/l) per refill*
- *Detect abnormal fuel consumption*

4.5 Issue Management

DRIVER can:

- *Report vehicle issue*

- *Track issue resolution status*

MECHANIC can:

- *Acknowledge issue report*
- *Mark issue as resolved*

4.6 Cost Tracking

FLEET_MANAGER can:

- *View cost summary per vehicle*
- *Generate monthly cost report*

---

5. Business Rules

- *Only FLEET_MANAGER can register vehicles*
- *Preventive maintenance interval: 5,000 km*
- *Fuel efficiency must be between 2-10 km/l (truck) or 10-20 km/l (car)*
- *Odometer reading must be strictly increasing*
- *CRITICAL issues must be resolved within 24 hours*
- *HIGH severity within 72 hours*
- *Maintenance cost cannot exceed vehicle value*
- *Monthly fuel budget per vehicle: ₹50,000 (configurable)*

Fuel Efficiency Calculation Rules

Previous Consumption = (CurrentOdometer - PreviousOdometer) / FuelAmount

Average Efficiency = TotalDistance / TotalFuel over last 30 days

Alert if Efficiency drops by > 20% from average

Validation Rules

- *Registration number format: XX00XX0000 (e.g., KA01AB1234)*
- *Odometer reading ≥ previous reading*
- *Fuel amount ≥ 1 liter*
- *Maintenance cost ≥ ₹0*
- *Issue severity must be provided*
- *Service interval cannot be less than 1,000 km*

---

6. REST API Requirements

- *Proper HTTP methods*
- *Clear DTOs*
- *Proper status codes*
- *Validation using annotations*
- *Structured error response*
- *Meaningful messages*

Sample APIs

User APIs:

```shell
POST /api/users

GET /api/users

GET /api/users?role=DRIVER

Vehicle APIs:


POST /api/vehicles

GET /api/vehicles

GET /api/vehicles/{id}

PUT /api/vehicles/{id}/odometer

PUT /api/vehicles/{id}/retire

Maintenance APIs:

POST /api/maintenance/schedule

POST /api/maintenance/records

PUT /api/maintenance/schedule/{id}/complete
```

GUVI

```
GET /api/maintenance/overdue

GET /api/maintenance/vehicle/{vehicleId}
```

```shell
Fuel APIs:

POST /api/fuel-logs

GET /api/fuel-logs/vehicle/{vehicleId}

GET /api/fuel-logs/efficiency/{vehicleId}

Issue APIs:

POST /api/issues

GET /api/issues/open

PUT /api/issues/{id}/acknowledge

PUT /api/issues/{id}/resolve

Cost APIs:

GET /api/costs/vehicle/{vehicleId}

GET /api/costs/summary

POST /api/costs/record
```

7. Database Requirements

Minimum Tables:

- *users*
- *vehicles*
- *maintenance_schedules*
- *maintenance_records*
- *fuel_logs*
- *issue_reports*
- *vehicle_costs*
- *vehicle_activity_logs*

:

- *Foreign key (vehicle → fleet_manager)*
- *Foreign key (maintenance_record → vehicle, mechanic)*
- *Foreign key (fuel_log → vehicle, driver)*
- *Foreign key (issue_report → vehicle, driver)*
- *Foreign key (vehicle_cost → vehicle)*
- *Enum mapping for roles, vehicle status, issue severity*

Must Include:

- *At least one JOIN query*
  Example: Fetch vehicle with maintenance history, fuel logs, and issue reports
- *At least one aggregate query*
  Examples:
  - *Total maintenance cost per vehicle per year*
  - *Average fuel efficiency by vehicle type*
  - *Number of critical issues per month*

---

8. GOOD TO HAVE

- *Global exception handling*
- *Pagination and sorting*
- *Unit tests for fuel efficiency calculation*
- *Swagger documentation*
- *Dashboard with fuel efficiency trends*
- *Maintenance reminder scheduler*
- *GPS integration for real-time odometer (mock)*
- *Email alert for overdue maintenance*
- *Cost comparison across vehicles*
- *Fuel theft detection (abnormal consumption)*

---

9. Non-Functional Expectations

These aspects will be observed during the hackathon:

- *Requirement understanding before coding*
- *Logical entity and API design*
- *Team discussion and task sharing*
- *Handling of edge cases*
- *Ability to explain design decisions*

Completeness is not mandatory. Approach and clarity are more important.

---

10. Hackathon Guidelines

Teams must discuss and plan before coding
Internet & AI usage is allowed
Code quality is preferred over quantity
Every team member is expected to contribute (write backend code)
Short walkthrough of solution at the end (5 minutes)

---

11. Evaluation Criteria

Candidates will be evaluated on:

- *Java and Spring Boot understanding*
- *Database and data modeling skills*
- *Problem-solving approach*
- *Team collaboration and communication*
- *Learning attitude and adaptability*
